

Conception, Implémentation, Comparaison et Analyse Critique de Modèles de Deep Learning

Données tabulaires (MLP) • Images (CNN) • Séquences (RNN / LSTM / GRU / Seq2Seq)

Réalisé par : **Lasri Mohamed Amine**

Groupe : **G1 — 4IADO**

Encadrante : **Mme. Zineb HIDILA**



Sommaire

Plan du rapport et de la soutenance



01

Introduction & problématique

Contexte, objectifs, organisation du rapport



03

Partie II — CNN

Vision par ordinateur & convolution



02

Partie I — MLP

Données tabulaires & ingénierie PyTorch



04

Partie III — RNN / LSTM / GRU / Seq2Seq

Séquences, traduction automatique



05

Discussion transversale

Adéquation données ↔ architecture



06

Conclusion & perspectives

Synthèse des résultats clés

Contexte & problématique

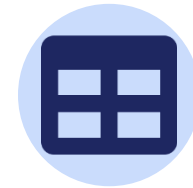
Le **deep learning** extrait automatiquement des représentations hiérarchiques à partir de données brutes, sans ingénierie manuelle de caractéristiques.

Le défi central : adapter l'architecture à la géométrie des données.

- Structure tabulaire → aucune géométrie spatiale
- Grille spatiale (images) → corrélation locale entre pixels
- Dépendances temporelles (séquences) → ordre et mémoire

PROBLÉMATIQUE

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image, séquentielle ?



Tabulaire

569 échantillons, 30 features
→ MLP



Image

60 000 images 32×32 RGB
→ CNN



Séquence

50 000 paires de phrases
→ RNN/LSTM/Seq2Seq

Objectifs du projet



Maîtriser PyTorch

nn.Module, paramètres, device, sauvegarde/rechargement des modèles.



Expérimenter rigoureusement

Comparaisons reproductibles : initialisation, architecture, hyperparamètres.



Analyser de façon critique

Relier théorie, implémentation et résultats expérimentaux.



Répondre à la problématique

Comment le DL adapte ses architectures à la structure des données ?

Organisation : chaque partie technique suit le même canevas





PARTIE I

MLP & Ingénierie PyTorch

Classification supervisée sur données tabulaires — Breast Cancer Wisconsin



MLP — Cadre théorique

Transformation d'une couche du MLP

$$h(l) = f(W(l) \cdot h(l-1) + b(l))$$

f = fonction d'activation non linéaire (ReLU)

Concepts PyTorch fondamentaux

- `nn.Module` : gestion auto. des paramètres, GPU, sérialisation
- Rétropropagation : règle de la chaîne sur la perte L
- `state_dict()` / `named_parameters()` : inspection & sauvegarde
- `device = cuda/cpu` : cohérence modèle $\leftrightarrow$ données



Stratégies d'initialisation testées

Gaussienne $N(0, 0.01^2)$ • Constante • Xavier $U(\pm\sqrt{6/(n_{in}+n_{out})})$

Architecture MLP (extrait)

```
class MLP(nn.Module):
    def __init__(self, in_dim,
                  hidden_dims, out_dim):
        layers = []
        prev = in_dim
        for h in hidden_dims:
            layers += [
                nn.Linear(prev, h),
                nn.BatchNorm1d(h),
                nn.ReLU(),
                nn.Dropout(0.3)]
            prev = h
        layers.append(
            nn.Linear(prev, out_dim))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)
```

MLP — Données & résultats

569

échantillons

30

features numériques

37/63 %

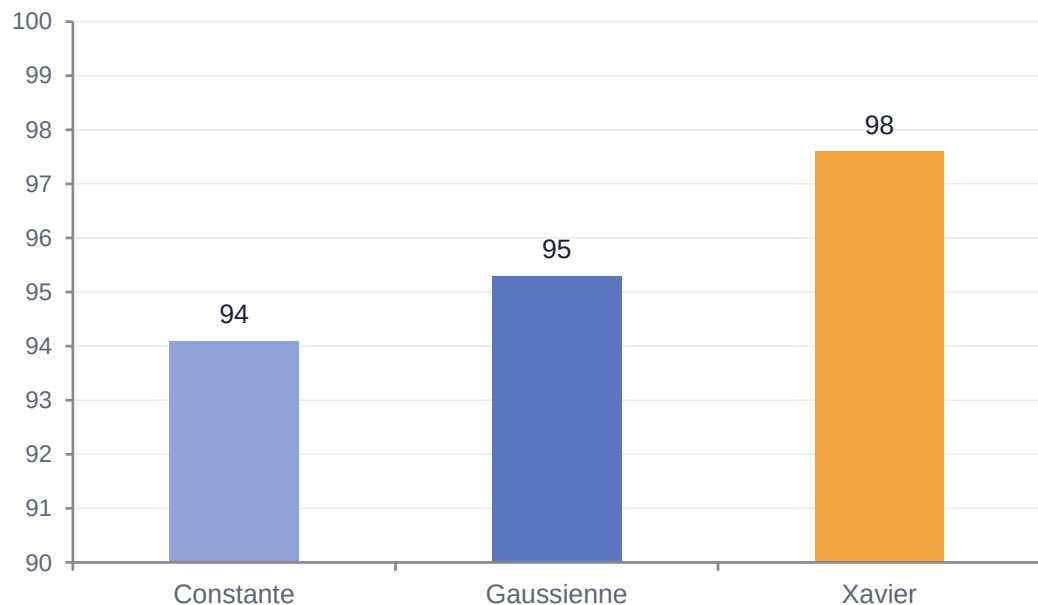
déséquilibre classes

97.6 %

accuracy (test)

Pipeline : split stratifié 70/15/15 → StandardScaler (fit train uniquement) → DataLoader

Impact de l'initialisation sur l'accuracy (validation)



Performances finales (test set, init. Xavier)

Classe	Précision	Rappel	F1-score
Malin (0)	0.971	0.962	0.966
Bénin (1)	0.979	0.985	0.982
Macro avg	0.975	0.974	0.974



41 époques (early stopping) — F1 macro = 0.974 sur le test set, validant l'architecture en entonnoir 128→64→32 + BatchNorm + Dropout.

MLP — Question de synthèse

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire, et quelles sont ses principales limites ?



Oui, lorsque la structure est non géométrique. 97.6 % accuracy, F1 macro 0.974 sur Breast Cancer Wisconsin.

Limites principales

01

Absence de biais inductif

Aucune hypothèse sur la structure → inadapté aux images/séquences

02

Sensibilité à la normalisation

StandardScaler indispensable (contrairement aux arbres de décision)

03

Interprétabilité limitée

Boîte noire malgré SHAP, comparé aux modèles linéaires

04

Données hétérogènes

Gradient boosting (XGBoost) souvent supérieur en présence de variables catégorielles

Conclusion : le MLP est une baseline solide et rapide pour les données tabulaires numériques.



PARTIE II

CNN & Vision par Ordinateur

Classification d'images et analyse convolutionnelle — CIFAR-10



CNN — Cadre théorique

Pourquoi le MLP est inadapté aux images

Une image $224 \times 224 \times 3 \rightarrow 150\,528$ neurones d'entrée $\rightarrow$ des centaines de millions de paramètres. Le MLP ignore la localité spatiale.

Localité

Champ récepteur local par neurone

Partage des poids

Même filtre partout $\rightarrow$ invariance translation

Hiérarchie

Bords $\rightarrow$ textures $\rightarrow$ objets

Corrélation croisée 2D

$$(X \star K)[i,j] = \sum_m \sum_n X[i+m, j+n] \cdot K[m,n]$$

Taille de sortie : $H_{out} = \lfloor (H - k_h + 2p)/s \rfloor + 1$

Exemple : entrée 4×4 , noyau 3×3 , stride 1, padding 0 $\rightarrow$ sortie 2×2

Pooling

Max-pooling : max des activations de la fenêtre (détecteur de saillance)

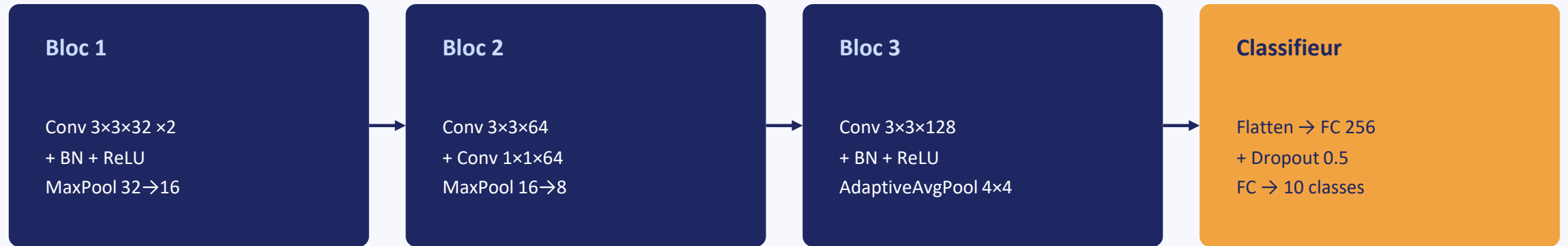
Average-pooling : moyenne des activations (lissage)

Vérification manuelle vs PyTorch

```
cross_corr2d_manual(X,K) vs F.conv2d(...)
→ différence max  $\approx 1e-6$  (erreur flottante)
```

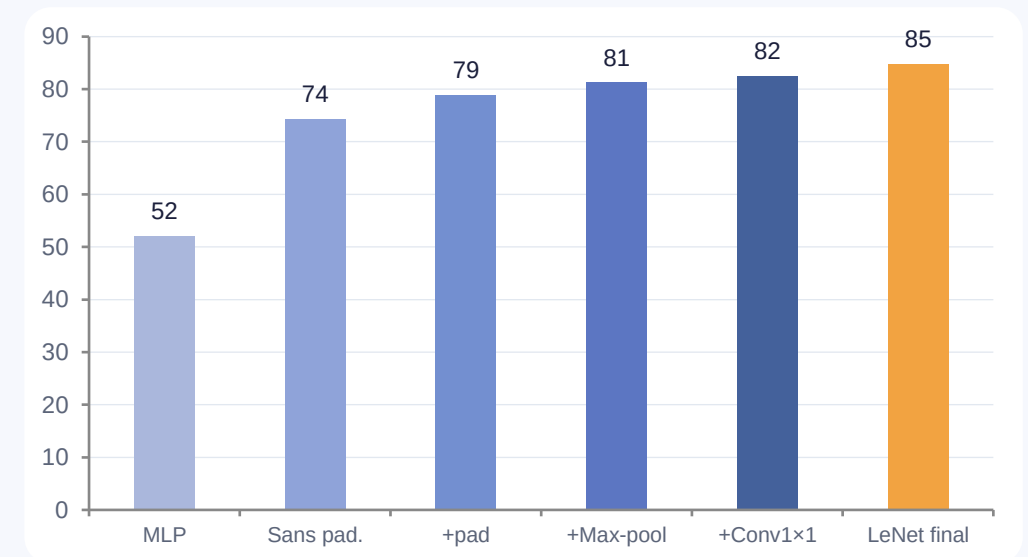
CNN — Architecture LeNetCIFAR

CIFAR-10 : 60 000 images 32×32 RGB, 10 classes équilibrées (50 000 train / 10 000 test)



Étude expérimentale des choix architecturaux

Configuration	Accuracy test	Paramètres
MLP baseline	52.1 %	0.84 M
CNN sans padding	74.3 %	0.23 M
CNN + padding=1	78.8 %	0.24 M
CNN + Max-pool	81.2 %	0.24 M
CNN + Conv 1×1	82.4 %	0.27 M
LeNetCIFAR (complet)	84.7 %	0.52 M



CNN — Résultats & synthèse

Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images ?

+32.6 pts

CNN vs MLP

Localité + partage des poids : 864 paramètres pour un filtre $3 \times 3 \times 32$ vs millions en fully-connected

+4.5 pts

Padding

Préserve les dimensions spatiales, traite correctement les bords de l'image

+1.6 pt

Max vs Avg-pool

Le max-pooling agit comme détecteur de la feature la plus saillante

+2.3 pts

Conv 1×1

Projection linéaire inter-canaux, capacité accrue sans coût spatial



Limite observée : LeNetCIFAR plafonne à 84.7 % sur CIFAR-10. Des architectures plus profondes avec connexions résiduelles (ResNet, VGG) atteignent 93–95 %, car la profondeur d'un CNN simple devient un goulot d'étranglement (gradient qui s'atténue lors de la rétropropagation).



PARTIE III

RNN, LSTM, GRU & Seq2Seq

Modélisation de séquences et traduction automatique — corpus fra-eng (Tatoeba)



Architectures récurrentes

RNN simple — équation d'état

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$$

$\|W_{hh}\| < 1 \rightarrow$ gradient s'évanouit • $\|W_{hh}\| > 1 \rightarrow$ explosion



Gradient clipping (c = 1.0)

$\hat{g} = \min(1, c/\|g\|) \cdot g$ — stabilise l'entraînement (cf. graphique de résultats)

LSTM — trois portes

Oubli $f_t = \sigma(W_f[h_{t-1}; x_t])$

Entrée $i_t = \sigma(W_i[h_{t-1}; x_t])$

Sortie $o_t = \sigma(W_o[h_{t-1}; x_t]) \rightarrow$ cellule c_t persistante

Modèle de langage & perplexité

$$P(x) = \prod_t P(x_t | x_1, \dots, x_{t-1}) \rightarrow \text{PPL} = \exp(-1/T \sum \log P)$$

RNN vs LSTM vs GRU

Critère	RNN	GRU	LSTM
États internes	h	h	h + c
Portes	0	2	3
PPL ↓	48.3	31.7	27.2
BLEU ↑	12.1	19.4	22.8
Stabilité	Faible	Bonne	Excellente

GRU — version simplifiée

$$z_t = \sigma(W_z[h_{t-1}; x_t])$$

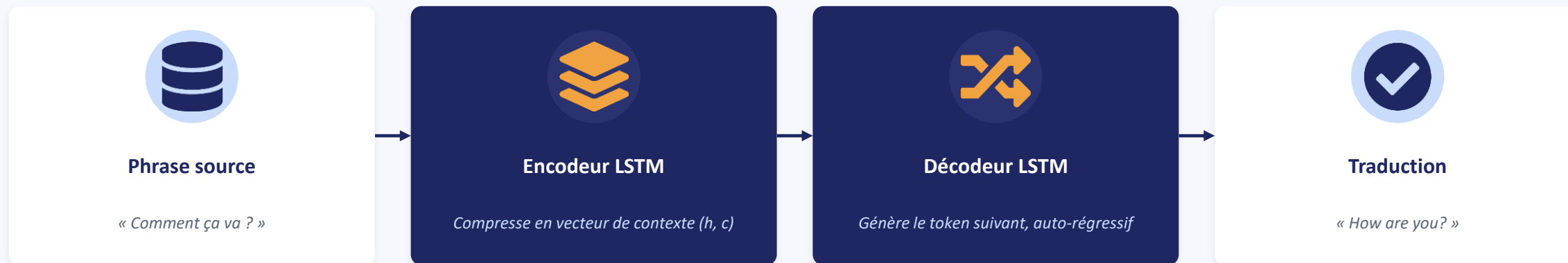
(mise à jour)

$$r_t = \sigma(W_r[h_{t-1}; x_t])$$

(réinitialisation)

$\rightarrow$ fusionne portes d'oubli et d'entrée du LSTM, 23 % moins de paramètres

Architecture Seq2Seq & décodage



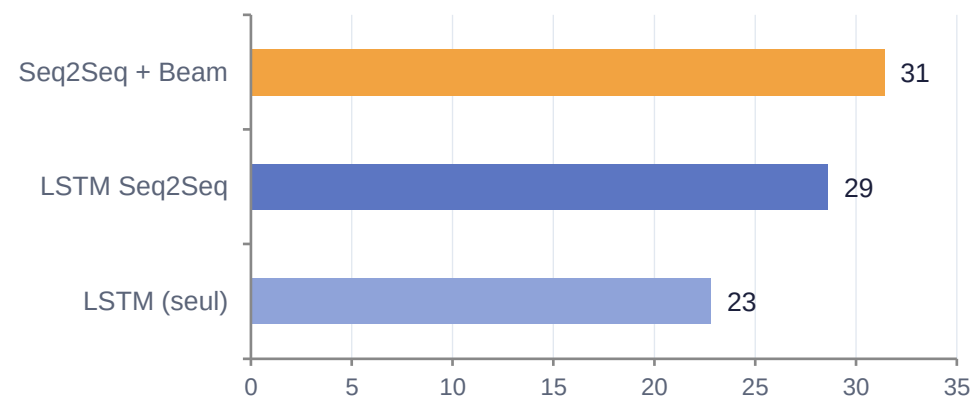
Teacher Forcing

Fournit le token correct (et non prédit) au décodeur à chaque étape pendant l'entraînement → accélère la convergence, mais crée un écart train/test (exposure bias).

Décodage glouton vs Beam Search

Le beam search explore $k=5$ hypothèses en parallèle et retient la séquence de score log-probabilité maximal, au lieu du choix glouton à chaque pas.

Gains BLEU sur fra-eng



Séquences — Synthèse

Comment justifier la progression RNN → LSTM/GRU → Seq2Seq pour la traduction ?

RNN simple

PPL 48.3

Disparition du gradient sur les longues dépendances

LSTM

PPL 27.2 (+44%)

Cellule de mémoire + 3 portes → gradient stable

Seq2Seq

BLEU 28.6 (+5.8)

Encodeur-décodeur : comprend la phrase avant de traduire

+ Beam Search

BLEU 31.4 (+2.8)

Explore k=5 hypothèses, évite l'optimum local du décodage glouton



Limite observée : les RNN/LSTM ont une complexité temporelle $O(T)$ non parallélisable. Les Transformers surpassent systématiquement les RNN sur corpus larges grâce à l'attention, mais exigent davantage de données et de ressources de calcul.

Discussion transversale

Comment le deep learning adapte-t-il ses architectures à la structure des données, et pourquoi le même paradigme supervisé se décline-t-il différemment selon la géométrie, la dépendance locale et la temporalité ?

Le biais inductif comme déterminant de la performance

Critère	MLP	CNN	RNN/LSTM	Seq2Seq
Biais inductif	Aucun	Localité spatiale	Temporalité	Séquence globale
Structure cible	Tabulaire	Grille 2D	Séquence	Séq. → Séq.
Parallélisme	Total	Total	Séquentiel	Séquentiel
Résultat clé	97.6 % acc.	84.7 % acc.	PPL 27.2	BLEU 31.4

Géométrie

Grille régulière des images → convolution

Dépendance locale

Pixels voisins corrélés → champ récepteur limité

Temporalité

Ordre des tokens crucial → état caché récurrent

Représentation

Données tabulaires sans structure → MLP adapté

Conclusion

Adapter l'architecture à la structure des données est le déterminant principal de la performance.



97.6 %

Accuracy MLP

Breast Cancer Wisconsin



84.7 %

Accuracy CNN

CIFAR-10, LeNetCIFAR



BLEU 31.4

LSTM Seq2Seq

+ Beam Search, fra-eng

Leçons méthodologiques

- Initialisation Xavier + Batch Normalization → convergence rapide
- Gradient clipping indispensable pour stabiliser les RNN
- Beam search améliore systématiquement la qualité du décodage
- Seeds fixes, early stopping, data leakage évité → reproductibilité



Perspective

Attention puis Transformers comme prochaine étape pour les tâches séquentielles.



Merci de votre attention

Questions & discussion



Lasri Mohamed Amine • Groupe G1 — 4IADO • Encadrante : Mme. Zineb HIDILA

EMSI Casablanca — Module Deep Learning — 2025-2026